

Universidad Nacional de Colombia

Facultad de Ingeniería

Taller 3: Programación discreta

Criptografía, grafos, álgebra de Boole, Shannon y computación cuántica

Estudiante: Sebastián Alexander Quistanchala Vallejo

Asignatura: Matemáticas Discretas I

Docente: Jhoan Sebastián Tenjo García

Modalidad: Individual

Lenguaje: Python 3, Google Colab

Bogotá D. C., 2026

Índice

1. Información general	2
1.1. Propósito	2
1.2. Repositorio y ejecución	2
2. Ejercicio 1: cifrado César	2
3. Ejercicio 2: RSA de juguete	3
4. Ejercicio 3: computación multipartita segura	3
5. Ejercicio 4: ruta más corta con Dijkstra	4
6. Ejercicio 5: cierre de una estación	5
7. Ejercicio 6: coloreo de grafos	5
8. Ejercicio 7: tablas de verdad y circuitos	6
9. Ejercicio 8: simplificación booleana	6
10. Ejercicio 9: entropía de Shannon	7
11. Ejercicio 10: simulador básico de un qubit	8

1. Información general

1.1. Propósito

El taller convierte conceptos de matemáticas discretas en programas verificables. Se desarrollaron diez ejercicios relacionados con criptografía clásica y modular, computación multipartita segura, algoritmos sobre grafos, álgebra de Boole, teoría de la información y simulación cuántica básica.

El código se encuentra en el cuaderno `Taller_3_Discretas.ipynb`. Cada ejercicio incluye una implementación general, ejemplos de ejecución y pruebas mediante instrucciones `assert`. Las librerías se emplearon para presentar tablas o gráficos, pero los algoritmos principales fueron implementados directamente.

1.2. Repositorio y ejecución

Repositorio: https://github.com/squistanchala-ai/Taller_3_Matematicas_Discretas.git

Para ejecutar el proyecto en Google Colab:

1. Abrir el archivo `Taller_3_Discretas.ipynb`.
2. Seleccionar un entorno de ejecución con Python 3.
3. Ejecutar las celdas en orden.
4. Introducir los datos solicitados por cada programa.
5. Ejecutar después la celda de pruebas correspondiente.

Las principales dependencias son `numpy`, `pandas`, `matplotlib`, `networkx` e `IPython.display`. También se utilizan módulos de la biblioteca estándar como `math`, `heapq`, `itertools`, `collections` y `secrets`.

2. Ejercicio 1: cifrado César

¿Qué problema resuelve? El programa cifra y descifra mensajes mediante un desplazamiento fijo del alfabeto. También permite probar los 26 desplazamientos posibles cuando se desconoce la clave.

¿Qué idea matemática utiliza? Cada letra se identifica con un elemento de $\mathbb{Z}_{26}$. Si x es la posición de una letra y k es el desplazamiento, el cifrado y el descifrado se definen por

$$E_k(x) = (x + k) \text{ mód } 26, \quad D_k(x) = (x - k) \text{ mód } 26.$$

El módulo 26 permite regresar a la letra A después de superar la Z. El descifrado usa el desplazamiento contrario porque ambas transformaciones deben ser inversas.

¿Cómo se ejecuta? Se ejecuta la celda del ejercicio y se selecciona una opción del menú: cifrar, descifrar con una clave conocida o realizar un ataque de fuerza bruta. El programa conserva espacios, números y signos de puntuación.

¿Qué pruebas se realizaron?

- HOLA UNAL, con $k = 3$, produce KROD XQDO.

- Un mensaje con minúsculas, espacios, números y puntuación se cifra y luego se recupera exactamente.
- Un desplazamiento $k = 29$ produce el mismo resultado que $k = 3$, pues $29 \equiv 3 \pmod{26}$.

¿Qué limitaciones tiene? La implementación principal usa únicamente las letras A–Z y a–z; la ñ y las vocales acentuadas no se transforman. El ataque de fuerza bruta muestra todas las posibilidades, pero no identifica automáticamente cuál es el mensaje correcto. Además, César no ofrece seguridad real porque solo tiene 26 claves posibles.

3. Ejercicio 2: RSA de juguete

¿Qué problema resuelve? El programa simula la generación de claves, el cifrado y el descifrado RSA para mensajes numéricos pequeños. Su propósito es mostrar la relación entre números primos, congruencias e inversos modulares.

¿Qué idea matemática utiliza? Dados dos primos distintos p y q , se calculan

$$n = pq, \quad \varphi(n) = (p-1)(q-1).$$

El exponente público e debe satisfacer $\gcd(e, \varphi(n)) = 1$. El exponente privado d es el inverso modular de e :

$$ed \equiv 1 \pmod{\varphi(n)}.$$

El inverso se obtiene mediante el algoritmo de Euclides extendido. El cifrado y el descifrado son

$$C \equiv M^e \pmod{n}, \quad M \equiv C^d \pmod{n}.$$

Las potencias se calculan con exponenciación modular rápida.

¿Cómo se ejecuta? Se introducen p , q , e y un mensaje entero M que cumpla $0 \leq M < n$. El programa valida los primos y el exponente, genera las llaves pública (e, n) y privada (d, n) , cifra el mensaje y comprueba que pueda recuperarse.

¿Qué pruebas se realizaron?

p	q	e	M	Resultado principal
61	53	17	65	$n = 3233$, $d = 2753$, $C = 2790$
11	13	7	42	$n = 143$, $d = 103$, $C = 81$

También se verificó que el programa rechaza un exponente no coprimo con $\varphi(n)$ y un valor de p que no sea primo.

¿Qué limitaciones tiene? Es una versión educativa: usa primos pequeños, trabaja con un solo entero y no implementa relleno criptográfico, codificación de texto ni generación segura de claves. Por estas razones no debe utilizarse para proteger información real.

4. Ejercicio 3: computación multipartita segura

¿Qué problema resuelve? El programa calcula la suma y el promedio de una lista de notas sin entregar la lista original a ninguno de los tres servidores simulados.

¿Qué idea matemática utiliza? Cada nota x se divide en tres partes aditivas aleatorias:

$$x \equiv s_1 + s_2 + s_3 \pmod{M}.$$

Se eligen aleatoriamente s_1 y s_2 , y se calcula

$$s_3 = (x - s_1 - s_2) \bmod M.$$

Cada servidor recibe una parte de cada nota y calcula únicamente su suma parcial. La suma total se reconstruye mediante

$$S = (S_1 + S_2 + S_3) \bmod M.$$

Se elige M mayor que la máxima suma posible para evitar que el resultado real se confunda con otro representante modular.

¿Cómo se ejecuta? Se introducen notas enteras entre 0 y 50 separadas por comas. El programa genera las partes secretas, presenta las tres sumas parciales y reconstruye solamente la suma total y el promedio.

¿Qué pruebas se realizaron?

- Para [40,35,50,25] se obtiene suma 150 y promedio 37.5.
- Para [0,50,20] se obtiene suma 70 y promedio 70/3.
- Para una única nota [45] se obtiene suma y promedio iguales a 45.
- Se verificó el rechazo de una nota fuera del intervalo y de una lista vacía.

¿Qué limitaciones tiene? Los tres servidores se simulan dentro del mismo proceso y no existe comunicación real por red. El modelo supone que los servidores no comparten sus partes y no contempla ataques, fallos de comunicación ni servidores maliciosos. Solo se aceptan notas enteras.

5. Ejercicio 4: ruta más corta con Dijkstra

¿Qué problema resuelve? El programa encuentra la ruta de menor costo entre dos lugares de una ciudad representada mediante un grafo ponderado no dirigido. Los pesos se interpretan como tiempos de viaje.

¿Qué idea matemática utiliza? Se implementó el algoritmo de Dijkstra. Inicialmente, $d(\text{origen}) = 0$ y las demás distancias son infinitas. Para cada arista (u, v) se aplica la relajación

$$d(v) = \min\{d(v), d(u) + w(u, v)\}.$$

Una cola de prioridad permite seleccionar el vértice no procesado con menor distancia conocida. Los predecesores guardados permiten reconstruir la ruta óptima.

¿Cómo se ejecuta? La red se define mediante una lista de adyacencia con 8 vértices y 14 aristas. Después se ingresan los nombres del origen y el destino. El programa entrega la distancia mínima y la secuencia de vértices de la ruta.

¿Qué pruebas se realizaron?

Origen	Destino	Distancia	Ruta obtenida
Portal	Centro	20	Portal – Calle 80 – Estadio – Universidad – Centro
Portal	Universidad	14	Portal – Calle 80 – Estadio – Universidad
Terminal	Museo	11	Terminal – Calle 26 – Universidad – Museo
Museo	Museo	0	Museo

También se comprobó que un destino inexistente produzca un mensaje de error.

¿Qué limitaciones tiene? Dijkstra requiere pesos no negativos. La red utilizada es sintética, estática y relativamente pequeña; no incluye tráfico en tiempo real, sentidos únicos ni cambios dinámicos de los pesos.

6. Ejercicio 5: cierre de una estación

¿Qué problema resuelve? El programa mide cómo cambian las rutas más cortas cuando se elimina un vértice de la red. En la prueba principal se cierra la estación **Universidad**.

¿Qué idea matemática utiliza? Se crea una copia del grafo, se elimina el vértice y todas sus aristas, y se vuelve a ejecutar Dijkstra para los mismos pares origen-destino. Para rutas que continúan conectadas se calcula

$$\Delta d = d_{\text{después}} - d_{\text{antes}}.$$

Si no existe un camino, la distancia se representa como infinita.

¿Cómo se ejecuta? El programa usa el grafo del ejercicio anterior, calcula cinco rutas antes del cierre, elimina **Universidad**, repite los cálculos y presenta una tabla comparativa.

¿Qué pruebas se realizaron?

Origen	Destino	Antes	Después	Diferencia	Estado
Portal	Centro	20	20	0	Sin cambio
Terminal	Centro	13	14	1	Aumentó
Calle 26	Centro	10	11	1	Aumentó
Terminal	Museo	11	12	1	Aumentó
Universidad	Centro	6	∞	—	Desconectado

Además, se comprobó que todas las conexiones de la estación fueran eliminadas y que el grafo original permaneciera sin modificaciones.

¿Qué limitaciones tiene? Se analiza un solo cierre a la vez y una lista fija de pares. El caso desconectado involucra directamente a la estación cerrada. No se realiza un análisis de múltiples cierres, capacidad, congestión ni robustez probabilística.

7. Ejercicio 6: coloreo de grafos

¿Qué problema resuelve? El programa asigna franjas de examen a materias con estudiantes en común. Cada materia es un vértice y cada conflicto es una arista; dos materias adyacentes no pueden compartir franja.

¿Qué idea matemática utiliza? Se implementó un coloreo voraz. Los vértices se procesan de mayor a menor grado y a cada uno se le asigna el menor color que no esté utilizado por sus vecinos ya coloreados. Después se verifica cada arista para comprobar que sus extremos tengan colores diferentes.

¿Cómo se ejecuta? El grafo de prueba contiene 10 cursos y 22 conflictos. La ejecución muestra el orden de procesamiento, la franja asignada a cada curso, los grupos por color y una visualización del grafo. **NetworkX** se usa únicamente para dibujar; la asignación de colores es propia.

¿Qué pruebas se realizaron?

- El grafo de cursos produjo una asignación válida con 4 franjas.
- El grafo completo K_5 utilizó 5 colores.
- El grafo bipartito completo $K_{3,3}$ utilizó 2 colores.
- Un grafo sin aristas utilizó un único color.

¿Qué limitaciones tiene? El coloreo voraz garantiza una asignación válida, pero no siempre encuentra el número cromático. El resultado puede cambiar si se modifica el orden de los vértices. Tampoco se incluyen restricciones adicionales como disponibilidad de docentes o salones.

8. Ejercicio 7: tablas de verdad y circuitos

¿Qué problema resuelve? El programa genera tablas de verdad y evalúa expresiones booleanas con las variables A, B, C, D y los conectivos AND, OR, NOT y XOR.

¿Qué idea matemática utiliza? Cada variable puede valer 0 o 1, por lo que cuatro variables producen $2^4 = 16$ combinaciones. Cada conectivo se implementa como una función booleana y representa una compuerta lógica. La tabla enumera todas las entradas y calcula la salida de la expresión.

¿Cómo se ejecuta? El menú permite mostrar todas las tablas o evaluar una expresión con una entrada concreta. Se incluyeron las tres expresiones solicitadas:

$$(A \wedge B) \vee (\neg C), \quad (A \oplus B) \wedge C, \quad (A \vee B) \wedge (\neg A \vee C),$$

y la expresión adicional

$$(A \wedge B) \oplus (C \vee D).$$

¿Qué pruebas se realizaron?

Expresión	A	B	C	D	Salida
$(A \wedge B) \vee \neg C$	1	1	0	0	1
$(A \oplus B) \wedge C$	1	0	1	0	1
$(A \vee B) \wedge (\neg A \vee C)$	1	0	0	1	0
$(A \wedge B) \oplus (C \vee D)$	1	1	0	0	1

También se verificó que cada tabla tenga 16 filas y que una entrada distinta de 0 o 1 sea rechazada.

¿Qué limitaciones tiene? Las expresiones se definen mediante funciones y no se incluye un analizador de expresiones arbitrarias escritas por el usuario. En las tres expresiones obligatorias, D no afecta el resultado, por lo que algunas filas se repiten respecto a la salida.

9. Ejercicio 8: simplificación booleana

¿Qué problema resuelve? El programa recibe una función booleana mediante sus minterminos y produce una expresión simplificada en forma de suma de productos. También demuestra la equivalencia entre la función original y la simplificada.

¿Qué idea matemática utiliza? Se implementó una versión pequeña del método de Quine–McCluskey. Los minterminos se convierten a binario y se combinan cuando difieren en exactamente una posición. Esa posición se reemplaza por un guion, indicando que la variable correspondiente

se elimina. Después se identifican implicantes primos, implicantes esenciales y una cobertura de todos los minterminos.

Dos expresiones se consideran equivalentes si producen la misma salida para todas las combinaciones de entrada.

¿Cómo se ejecuta? Se seleccionan 3 o 4 variables y se escriben los minterminos separados por comas. El programa muestra los implicantes, la expresión simplificada y una tabla que compara ambas funciones.

¿Qué pruebas se realizaron? El caso principal fue

$$F(A, B, C) = \Sigma m(1, 3, 5, 7).$$

Todos esos minterminos tienen $C = 1$, por lo que el patrón combinado es -1 y la simplificación es

$$F = C.$$

También se probaron una función con dos grupos, las funciones constantes 0 y 1, un caso de cuatro variables y un mintermino fuera del rango permitido.

¿Qué limitaciones tiene? La implementación acepta solamente tres o cuatro variables y no incluye condiciones “no importa”. La búsqueda de una cobertura mínima prueba combinaciones de implicantes, por lo que su costo puede crecer de forma exponencial. No está diseñada para funciones grandes.

10. Ejercicio 9: entropía de Shannon

¿Qué problema resuelve? El programa mide la incertidumbre de un texto a partir de la frecuencia de sus símbolos y compara dos mensajes para determinar cuál tiene mayor entropía.

¿Qué idea matemática utiliza? Para cada símbolo se calcula

$$p_i = \frac{f_i}{N},$$

donde f_i es su frecuencia y N es la longitud del mensaje. La entropía de Shannon es

$$H = - \sum_i p_i \log_2(p_i),$$

y se expresa en bits por símbolo. Un mensaje repetitivo es más predecible y tiene menor entropía; uno con símbolos variados y distribuidos uniformemente tiene mayor entropía.

¿Cómo se ejecuta? Se introducen dos textos. El programa considera letras, espacios, números y signos como símbolos, calcula sus frecuencias y probabilidades, muestra las tablas y genera gráficos de comparación.

¿Qué pruebas se realizaron?

Mensaje	Entropía
AAAAAAAAAA	0 bits/símbolo
AABB	1 bit/símbolo
ABCD	2 bits/símbolo
ABCDEFGH	3 bits/símbolo

También se comprobó que los espacios se cuenten como símbolos y que el texto vacío sea rechazado.

¿Qué limitaciones tiene? Se calcula una entropía empírica de orden cero: solo importa la frecuencia individual y no el orden ni la dependencia entre símbolos. El análisis distingue mayúsculas de minúsculas. La entropía no es un algoritmo de compresión y los gráficos pueden saturarse si el texto contiene demasiados símbolos diferentes.

11. Ejercicio 10: simulador básico de un qubit

¿Qué problema resuelve? El programa simula un qubit, la aplicación de las compuertas X, Z y H, el cálculo de probabilidades y 1000 mediciones.

¿Qué idea matemática utiliza? El estado se representa mediante

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

Las compuertas se aplican mediante multiplicación matricial:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

La regla de Born establece

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2.$$

Las mediciones se generan aleatoriamente con esas probabilidades.

¿Cómo se ejecuta? Se selecciona el estado inicial $|0\rangle$ o $|1\rangle$ y se introduce una secuencia como H, X, Z. El programa muestra el estado después de cada compuerta, las probabilidades teóricas, las frecuencias observadas en 1000 mediciones y un gráfico comparativo.

¿Qué pruebas se realizaron?

- $X|0\rangle = |1\rangle$.
- $H|0\rangle$ produce $P(0) = P(1) = 0.5$ y frecuencias observadas cercanas a 50 %.
- $HH|0\rangle = |0\rangle$, salvo redondeo numérico.
- $Z|1\rangle = -|1\rangle$; la fase cambia, pero la probabilidad de medir 1 continúa siendo uno.
- Se verificó el rechazo de una compuerta no implementada.

¿Qué limitaciones tiene? Se simula únicamente un qubit ideal en un computador clásico. No se incluyen entrelazamiento, múltiples qubits, decoherencia, ruido, errores de compuerta ni conexión con hardware cuántico. Las mediciones son muestras pseudoaleatorias de una distribución calculada clásicamente; no son ejecuciones en un procesador cuántico real.